

One More Example of Creating Loops

1. Create a program that can binary-search for x (where $b[0] \leq x \leq b[n-1]$ and $n = \text{size}(b)$) in a non-empty sorted array b with no duplicated numbers. In the postcondition, let L be the index of x if x is in b , $0 \leq L < n$.

- How does binary search work?

We let L and R be the left (inclusive) and right (exclusive) boundary of the “search range”: elements in this range are candidate for target x . We know that the idea of binary search is to “halve” the search range after each iteration (by letting mid-point m be the new L or R in the next iteration) until the target x is found or the search range is shrunk to size 0 (or 1). We need to be careful while guaranteeing that the search range shrinks after each iteration: because when close to the end of the loop, it is possible that $R = L + 1$, then $m = \frac{L+R}{2} = L$ which might lead to divergence. Here let’s look one way to solve this problem (we terminate the loop when the search range has size 1):

We assign $L := m$ or $R := m$ in each iteration, then we need to terminate with $R = L + 1$ (which means there is only $b[L]$ in the search range) if x is not found. Because R can be $L + 1$ and L can be $n - 1$, we need to artificially define $b[n] = b[n - 1] + 1$.

(As an aside, another design can be let $L := m + 1$ or $R := m$ after each iteration. Think about how to create a loop using this design.)

- The precondition of the program can be $\text{Sorted}(b) \wedge \text{size}(b) = n > 0 \wedge b[0] \leq x \leq b[n-1]$, where $\text{Sorted}(b) \equiv \forall 0 \leq k < \text{size}(b) - 1. b[k] < b[k+1]$. Since $\text{Sorted}(b)$ is always true during the program and our searching procedure doesn’t change it; we will only show it in the precondition and omit it everywhere else.
- In the postcondition, to show whether we find x , we introduce a Boolean variable *found*. We say *found* only if we have $b[L] = x$ at the end. The postcondition of the program can be written as $q \equiv (0 \leq L < n) \wedge (b[L] \leq x < b[L+1]) \wedge (\text{found} \rightarrow b[L] = x)$.
- For loop invariant:
 - ❖ Dropping off either conjunct **doesn’t look promising**. (The dropped conjunct will be $\neg B$.)
 - ✚ $\neg(0 \leq L < n) \Leftrightarrow L < 0 \vee L \geq n$
 - ✚ $\neg(b[L] \leq x < b[L+1]) \Leftrightarrow x < b[L] \vee x \geq b[L+1]$ where L is the left-end of the search range
 - ✚ $\neg(\text{found} \rightarrow b[L] = x) \Leftrightarrow \text{found} \wedge b[L] \neq x$
 - ❖ Replacing expression $L + 1$ by a variable R can be a good idea. The range of R should be $L + 1 \leq R \leq n$, and we can end the loop with either $R = L + 1$ or *found*.
 - ❖ In the end, we can try to use loop invariant $p \equiv (0 \leq L < R \leq n) \wedge (b[L] \leq x < b[R]) \wedge (\text{found} \rightarrow b[L] = x)$
 - ❖ At the same time, we find that we should try to use loop condition $B \equiv \neg \text{found} \wedge R \neq L + 1$.

- For the bound expression: we will increase L or decrease R in each iteration, and loop invariant implies $R > L$, so we can use $R - L$ as the bound expression.
- Then we can come up with the following partial program:

```

{Sorted(b) ∧ size(b) = n > 0 ∧ b[0] ≤ x ≤ b[n - 1] < b[n]}
...
{inv p ≡ (0 ≤ L < R ≤ n) ∧ (b[L] ≤ x < b[R]) ∧ (found → b[L] = x)} {bd R - L}
while ¬found ∧ R ≠ L + 1 do
  {p ∧ ¬found ∧ R ≠ L + 1 ∧ R - L = t₀}
  m := (L + R) ÷ 2;
  {p₁ ≡ p ∧ ¬found ∧ R ≠ L + 1 ∧ R - L = t₀ ∧ m = (L + R) ÷ 2}    # forward assignment
  if b[m] = x then
    {p₁ ∧ b[m] = x} found := T; L := m {p ∧ R - L < t₀}
  else
    {p₁ ∧ b[m] ≠ x} ... {p ∧ R - L < t₀}
  fi
  {p ∧ R - L < t₀}
od
{0 ≤ L < R ≤ n ∧ (b[L] ≤ x < b[R]) ∧ (found → b[L] = x) ∧ (found ∨ R = L + 1)}
{q ≡ 0 ≤ L < n ∧ (b[L] ≤ x < b[L + 1]) ∧ (found → b[L] = x)}

```

- There are still gaps in the program and we don't have a full proof outline yet.
 - ❖ To get **inv p** from the precondition, we need to initialize values of L, R and $found$.
 - ❖ The true branch lacks proof: so, we need to add forward or backward assignments between statements. We omit the proof here.
 - ❖ For the false branch: we can use another conditional statement to assign $L := m$ or $R := m$. Then we have the following partial proof outline under total correctness:

```

{Sorted(b) ∧ 1 ≤ n = size(b) ∧ b[0] ≤ x ≤ b[n - 1] < b[n]}
L := 0; R := n; found := F;
{1 ≤ n = size(b) ∧ b[0] ≤ x ≤ b[n - 1] < b[n] ∧ L = 0 ∧ R = n ∧ found = F}
{inv p ≡ 0 ≤ L < R ≤ n ∧ (b[L] ≤ x < b[R]) ∧ (found → b[L] = x)} {bd R - L}
while ¬found ∧ R ≠ L + 1 do
  {p ∧ ¬found ∧ R ≠ L + 1 ∧ R - L = t₀}
  m := (L + R) ÷ 2;
  {p₁ ≡ p ∧ ¬found ∧ R ≠ L + 1 ∧ R - L = t₀ ∧ m = (L + R) ÷ 2}
  if b[m] = x then
    {p₁ ∧ b[m] = x} found := T; L := m; {p ∧ R - L < t₀}
  else
    {p₁ ∧ b[m] ≠ x} if b[m] > x then R := m else L := m fi {p ∧ R - L < t₀}
  fi
  {p ∧ R - L < t₀}
od
{p ∧ (found ∨ R = L + 1)}
{q ≡ 0 ≤ L < n ∧ (b[L] ≤ x < b[L + 1]) ∧ (found → b[L] = x)}

```

Substitution in Arrays

- We haven't discussed how to understand array assignments yet. If we assign an expression to $b[e]$, the index e can only be evaluated at runtime; which can lead to the following problem.

Assuming $b[j]$ and $b[k]$ are safe and $j \neq k$, then what is $wp(b[j] := b[j] + 1, b[k] < b[j])$?

- Some might say it is $b[k] < b[j] + 1$ but what if k and j evaluated to the same integer at runtime? Will we only substitute $b[j]$ with $b[j] + 1$? Or will we substitute both $b[j]$ and $b[k]$ with $b[j] + 1$ (and if so, we will get the weakest precondition being $b[j] + 1 < b[j] + 1 \Leftrightarrow F$)?
- We need to figure out how to understand substitutions in arrays $(e)[e_1 / b[e_0]]$ first.
- 2. How to understand $(b[m])[6 / d[2]]$ where m is a variable or named constant?
 - If b and d are different arrays ($b \neq d$), then this is simple: there will be no expression $d[2]$ in array $b[m]$ then $(b[m])[6 / d[2]] \equiv b[m]$.
 - If b and d are the same array ($b \equiv d$), then we need to consider whether $m = 2$: if $m = 2$ then $(b[m])[6 / d[2]] \equiv 6$ or else it $\equiv b[m]$.
- 3. How to understand $(b[e])[6 / d[2]]$ where e is an expression?
 - If b and d are different arrays, then $b[e] \neq d[2]$ and we need to look recursively into e since expression $d[2]$ might appear in e ; then, $(b[e])[6 / d[2]] \equiv b[e[6 / d[2]]]$.
 - If b and d are the same array, we need to evaluate e . If e is evaluated to 2 at run time, then $(b[e])[6 / d[2]] \equiv 6$, or else we will look recursively into e like in the above case.
- Here we give the definition of syntactic substitution in arrays:

$$(b[e_2])[e_1 / b[e_0]] \equiv \text{if } e'_2 = e_0 \text{ then } e_1 \text{ else } b[e'_2] \text{ fi, where } e'_2 \equiv (e_2)[e_1 / b[e_0]].$$
 - This definition covers all cases in Example 1 and 2 while the substitution happens in the same array:
 - When e_2 is a named constant, aka $e_2 \equiv k$, then we get $e'_2 \equiv k[e_1 / b[e_0]] \equiv k$, and then $(b[k])[e_1 / b[e_0]] \equiv \text{if } k = e_0 \text{ then } e_1 \text{ else } b[k] \text{ fi}$.